

Context On Tabs (COTs)

A visual-context research engine for the browser.

1. Introduction

COTs (Context-On-Tabs) is an interactive browser extension that gives users autonomy over visual, graphical context tabs. Users highlight a piece of content, text, image, link and COTs spawns a floating AI-powered panel tethered to that selection. Each panel is an independent conversation with an LLM, and the set of all panels forms a persistent visual graph that captures the user's research trajectory.

The core hypothesis:

Does contextual visual reasoning improve browser research workflows?

If the answer is yes, then semantic intelligence and multi-agent orchestration become natural future expansions.

2. Vision & Philosophy

2.1 Core Principles

- **Visual persistence:** every AI interaction stays visible and spatially arranged so the user can see the shape of their research.
- **Source anchoring:** no response floats free; every panel is edge-connected to its source highlight.
- **Progressive complexity:** start with highlights and panels, grow into semantic clustering and multi-agent orchestration.
- **User agency:** all panels are movable, resizable, minimizable, and closeable. The user owns the layout.

2.2 Research-First Design

COTs treats web browsing as a *research workflow*, not a sequence of isolated pages. It is designed for:

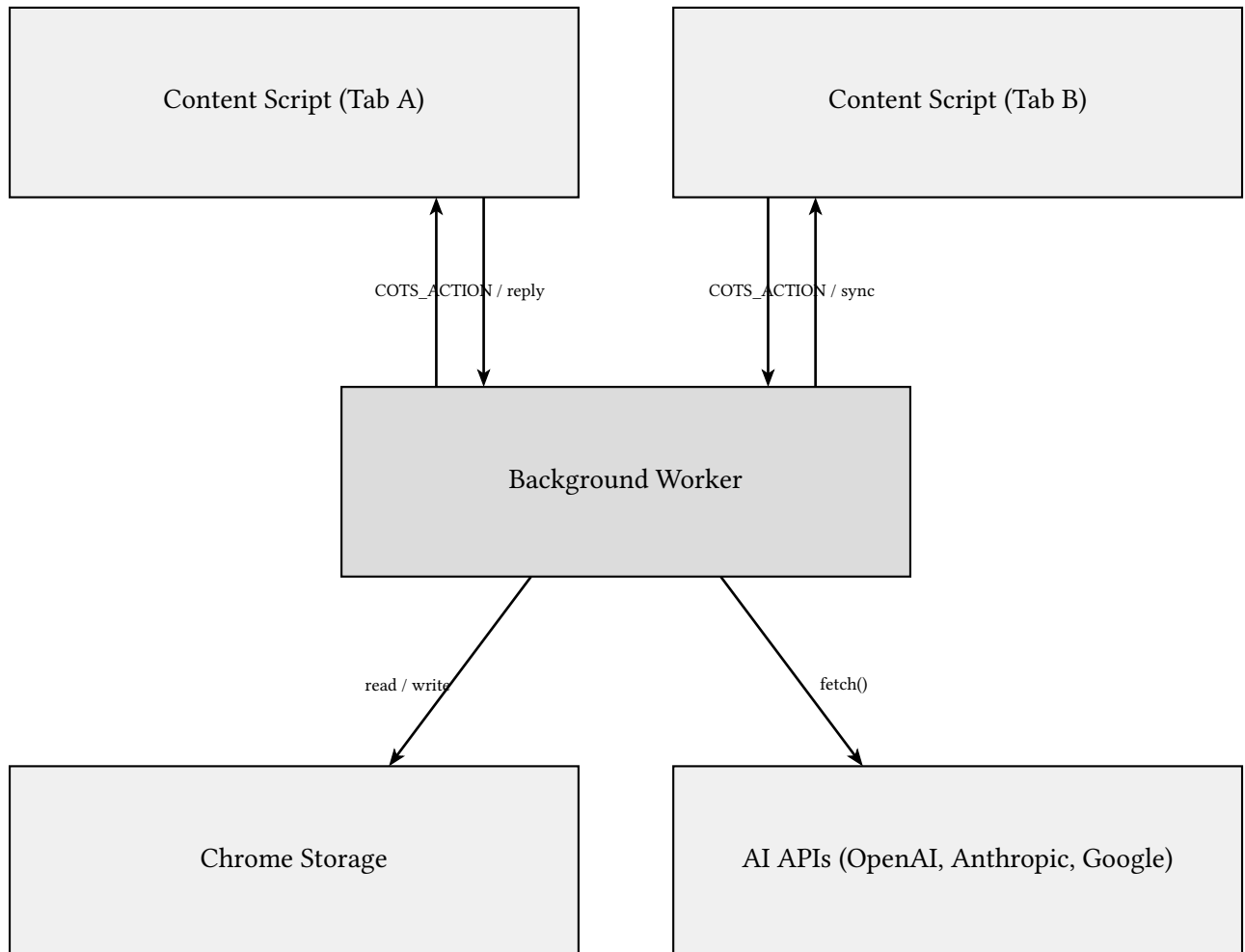
- Literature reviews across multiple open tabs
- Comparative analysis of sources
- Exploratory reading where questions branch into sub-questions
- Knowledge synthesis where the user assembles a map of what they have learned

2.3 Influences & Related Work

- **Knowledge graphs** (Ji et al., 2021). The graph structure underlying COTs nodes and edges.
- **LLM-assisted knowledge graph visualization** (Li et al., 2025). How visualisation affects trust, exploration, and workflows.
- **Zettelkasten / Roam / Obsidian**. The note-taking graph metaphor adapted to *live* browser context.
- **Jupyter notebooks**. The idea of cells with independent state, translated into floating panels.

3. Architecture Overview

3.1 System Diagram



The background worker is the central state hub. Content scripts never write to storage directly, they send `COTS_ACTION` messages. The background worker persists, then broadcasts the updated state to all active tabs via `browser.tabs.sendMessage` (using `webextension-polyfill`). This ensures cross-tab consistency without page reloads.

3.2 Extension Surfaces

Surface	File	Purpose
Content script	<code>src/contents/cots-overlay.tsx</code>	Selection detection, panel rendering, drag/resize, SVG edge rendering, group UI, cross-tab sync
Background worker	<code>src/background.ts</code>	Central state hub, message routing, AI request dispatch, cross-tab broadcast, group summarization

Surface	File	Purpose
Popup	src/pages/Popup.tsx	Graph summary, group management, clear-reset control
Options	src/pages/Options.tsx	AI provider config, UI defaults, theme
Storage lib	src/lib/storage.ts	Browser storage CRUD for graph, groups, settings
Types lib	src/lib/types.ts	All shared TypeScript types including groups and cross-tab messages
AI lib	src/lib/ai.ts	Provider-specific API callers
ID lib	src/lib/id.ts	UUID generation utility

3.3 Tech Stack

Layer	Technology
Framework	Plasmo 0.90.5 (React 18, TypeScript, cross-browser build targets)
Browser API	webextension-polyfill by Mozilla (write browser.*, run on Chrome + Firefox)
UI	React, TailwindCSS, Lucide icons
Bundler	Parcel 2.9.3 (pinned via overrides)
Storage	Browser storage API (storage.local)
AI	OpenAI, Anthropic, Google Gemini, or custom OpenAI-compatible
Manifest	MV3 (Chromium) / MV2 (Firefox)

4. Data Model

4.0 Cross-Tab Messaging

Content scripts and the background worker communicate via a typed action/broadcast system:

Actions (content script → background):

Type	Payload	Effect
PANELS_UPDATED	{ nodes, edges }	Persist graph, broadcast to tabs
PANEL_DELETED	panelId	Remove panel + edges, broadcast
GROUPS_UPDATED	CotsGroup[]	Persist groups, broadcast
GROUP_DELETED	groupId	Remove group, unlink nodes, broadcast full state
SUMMARIZE_GROUP	{ groupId }	Generate AI summary for group, broadcast result

Type	Payload	Effect
FULL_STATE_REQUEST	—	Return { graph, groups } to requester
SYNC_REQUEST	—	Broadcast full state to all tabs

Broadcasts (background → all content scripts):

Type	Payload	Trigger
FULL_STATE_SYNC	{ graph, groups }	Group deletion, manual sync
GRAPH_UPDATED	CotsGraph	Any panel create/update/delete
GROUPS_UPDATED	CotsGroup[]	Group create/update
GROUP_SUMMARY	{ groupId, summary }	AI summarization complete

This architecture ensures all tabs stay synchronized without requiring manual reloads.

4.1 Core Types

SelectionRect { x, y, width, height, top, right, bottom, left }

Represents the bounding box of the user's text selection, stored in absolute page coordinates (including scroll offset).

SelectionAnchor { text, pageTitle, pageUrl, rect: SelectionRect, createdAt }

Captures everything needed to identify and reconnect to a source highlight.

```
CotsNodePanel {                                // persisted panel geometry
  sourceId, x, y, width, height
  minimized, maximized: boolean
  groupId?: string
}
```

```
CotsNode {
  id: string
  kind: "selection" | "ai-panel" | "group-summary"
  label: string
  anchor?: SelectionAnchor           // present for all kinds
  panel?: CotsNodePanel             // only on ai-panel / group-summary nodes
  groupId?: string                  // optional group membership
  tabId?: number                    // originating tab
  tabUrl?: string                   // page where the panel was created
}
```

A *selection node* represents a piece of highlighted content. An *ai-panel node* represents one floating response panel. A *group-summary node* is a special panel showing an AI-generated correlation summary of a group. Both ai-panel and group-summary nodes point back to a SelectionAnchor.

CotsEdge { id, sourceId, targetId, label: string }

A directed edge from a selection node to its ai-panel node. The label describes the relationship (e.g. "context").

CotsGraph { nodes: CotsNode[], edges: CotsEdge[], updatedAt: string }

The full persisted state of the user's research session.

4.2 Groups

```
CotsGroup {
  id: string
  label: string
  nodeIds: string[]           // panel node IDs belonging to this group
  summary?: string           // AI-generated correlation summary
  createdAt: string
  updatedAt: string
}
```

Groups are first-class entities alongside the graph. A panel is assigned to a group via its `groupId` field (on both `CotsNode` and `CotsNodePanel`). Groups are persisted under the `cots.groups` storage key and are synchronized across all tabs. The group summary is generated by sending all member panel texts to the AI provider with a synthesis prompt.

4.3 Graph Invariants

- Every ai-panel node has exactly one incoming edge from its `sourceId`.
- A selection node can have many outgoing edges (multiple panels from one highlight).
- Deleting a panel removes its node and the connecting edge; the source node is retained in case other panels reference it.
- Clearing the graph removes all nodes and edges.

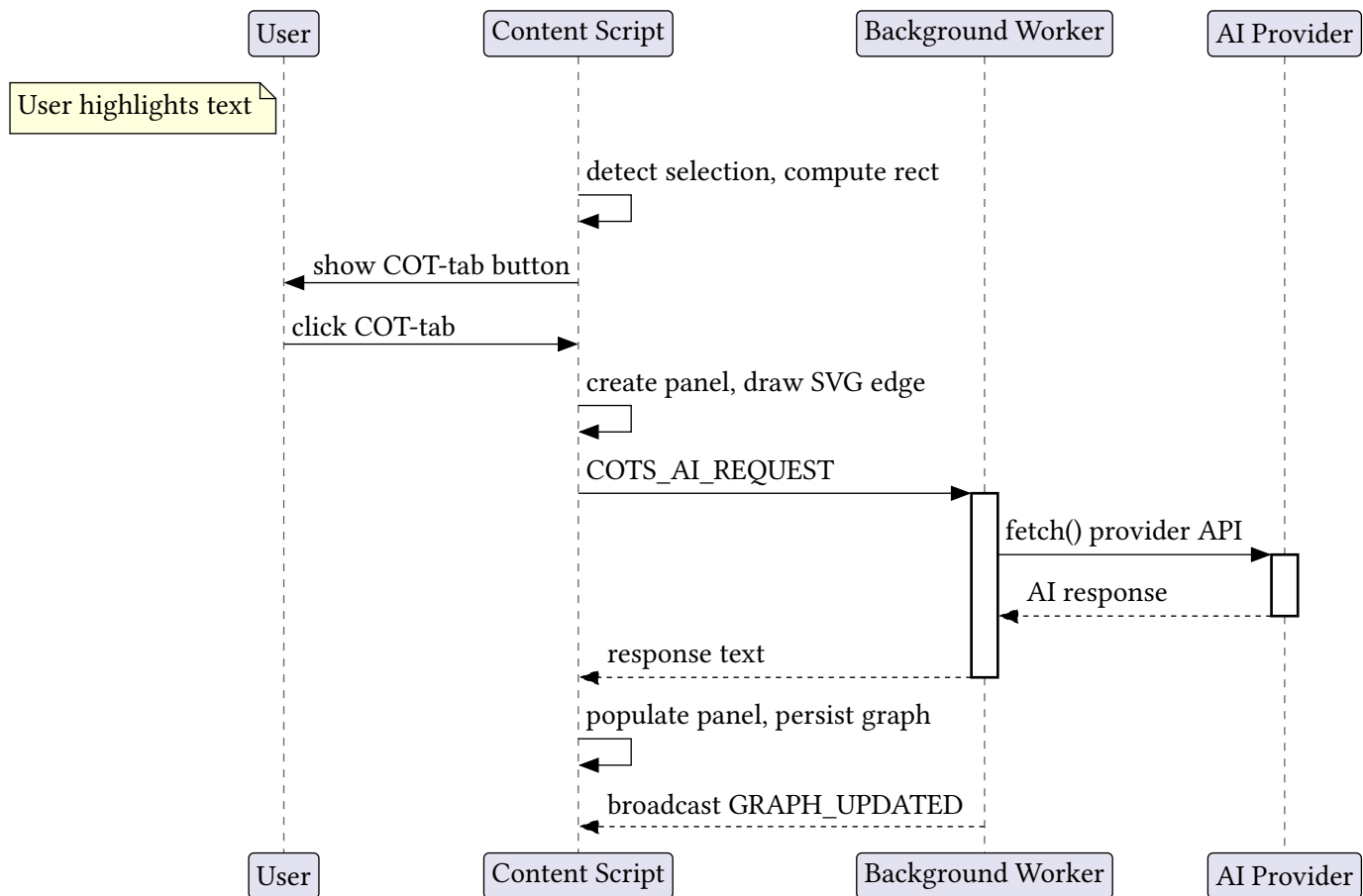
4.4 Settings Schema

```
AiProvider = "openai" | "anthropic" | "google" | "custom"
```

```
Settings {
  aiProvider, openaiApiKey, openaiModel
  anthropicApiKey, anthropicModel
  googleApiKey, googleModel
  customEndpoint, customModel
  systemPrompt: string           // default: research-assistant prompt
  maxTokens: number              // default: 1000
  temperature: number            // default: 0.7
  enableFollowUp: boolean         // default: true (show follow-up input)
  theme: "dark" | "light" | "auto"
  panelWidth, panelHeight        // default: 360 x 300
}
```

5. User Flow

5.1 Primary Flow



5.2 Secondary Flows

Follow-up questions:

- The user types into the input at the bottom of any panel.
- The question and conversation history are sent as a new COTS_AI_REQUEST.
- The response is appended to the panel's conversation history.

Panel manipulation:

- Drag by the header to reposition (pointer events with capture).
- Resize via the corner handle (clamped to 280–680 × 180–640).
- Minimize (collapses to header only).
- Maximize (expands to viewport minus 32 px padding).
- Close (removes the panel and its graph edge).

Session restoration:

- On page load, content script requests full state from background.
- Nodes with kind: "ai-panel" are deserialized into PanelState objects.
- Panels reappear at their saved positions with the source selection displayed.
- Follow-up conversation history is not persisted in the current version.

Graph reset:

- The popup sends PANELS_UPDATED with empty nodes/edges.
- All storage keys are cleared; the overlay resets via broadcast.

6. Panel System Specification

6.1 PanelState (Internal Runtime Type)

```
PanelState {
  id, sourceId, title: string
  selection: SelectionAnchor
  x, y, width, height: number
  minimized, maximized: boolean
  loading: boolean
  response: string // initial AI response text
  conversationHistory: Array<{ role: "user" | "assistant", content: string }>
  settings: Settings
  groupId?: string // assigned group membership
  kind: CotsNodeKind // "ai-panel" | "group-summary"
}
```

This is the runtime representation used by React state. It is *not* persisted directly; instead it is mapped to/from CotsNode[] + CotsEdge[] for storage.

6.2 Drag System

- Uses onPointerDown / onPointerMove / onPointerUp (pointer events, not mouse events) for low-latency dragging.
- setPointerCapture keeps the drag active even when the pointer leaves the element.
- The DragState union discriminates between "move" and "resize" operations.
- During a *move*, the panel's x/y is offset by the delta from the initial pointer position, adjusted for scroll.
- During a *resize*, width/height are clamped to min/max bounds.

6.3 Edge (SVG) Rendering

- Edges are rendered as a single <svg> overlay at fixed position covering the full viewport.
- Each edge is a <line> from (selection.right, selection.top + height/2) to (panel.x, panel.y + 28) (header midpoint).
- Coordinates are converted from absolute page coordinates to viewport coordinates by subtracting window.scrollX/Y.
- A viewportVersion counter increments on scroll and resize, triggering edge recalculations without full re-renders.
- Default edge style: stroke="#1e3a5f" strokeWidth="1.5" strokeDasharray="4 4" opacity="0.5".
- Grouped panel edges: color is derived from the group ID via hash (HSL), line is slightly thicker (2px) with a tighter dash pattern (2 2). A floating group label badge appears above the topmost panel in each group.

6.4 Group UI

- Each panel header has a group selector button (folder icon) that opens a dropdown.
- Dropdown shows: current group (if any), "Remove from group", "Create new group", and all other groups.
- "Create new group" opens a dialog with multi-select checkboxes for all panels.
- Grouped panels have a color dot next to their title, matching their edge color.

- Group name badges float above the topmost panel in each group, showing the label and member count.
- A summary tooltip can be toggled on the badge showing the AI-generated correlation summary.

7. AI Integration

7.1 Provider Support

Provider	Endpoint	Auth	Models (examples)
OpenAI	POST /v1/chat/completions	Bearer \$OPENAI_API_KEY	GPT-4o, GPT-4o-mini, o1
Anthropic	POST /v1/messages	x-api-key header	Claude 3.5 Sonnet, Haiku
Google	POST /v1/models/{model}:generateContent	?key= query param	Gemini 1.5 Pro/Flash, 2.0 Flash
Custom	User-defined URL	User-defined	OpenAI-compatible

7.2 Request Format

```
AiRequest {
  selection: SelectionAnchor    // page context + highlighted text
  prompt?: string              // optional user prompt; default = "Explain this
context..."
  conversationHistory?: Array<{ role, content }>
}
```

The background worker enriches the request by wrapping the selection context around the user prompt:

```
System: {settings.systemPrompt}
```

```
User:
```

```
Page: {selection.pageTitle}
URL: {selection.pageUrl}
Highlighted context:
{selection.text}
```

```
User prompt:
```

```
{prompt ?? "Explain this context and suggest next research branches."}
```

7.3 Response Format

```
AiResponse {
  text: string
  model: string
  conversationId?: string    // reserved for future session tracking
}
```

7.4 Error Handling

- Missing API keys return a friendly message (not a thrown error) so the user sees guidance.
- HTTP errors are parsed for provider-specific error messages.
- The extension never exposes API keys outside `chrome.storage.local` (keyed by the user in Options).

7.5 Group Summarization

When the user requests a correlation summary for a group:

1. All panel texts in the group are collected and numbered.
2. A synthesis prompt is constructed: *“Synthesize the following research excerpts into a coherent summary. Identify common themes, contradictions, and connections between them.”*
3. The request is sent through the standard AI pipeline.
4. The result is stored in `CotsGroup.summary` and broadcast as `GROUP_SUMMARY` to all tabs.
5. The summary is displayed in a tooltip on the group badge and in the popup.

8. Persistence

8.1 Storage Keys

Key	Type	Content
<code>cots.graph</code>	<code>CotsGraph</code>	Serialized graph (nodes + edges + timestamp)
<code>cots.groups</code>	<code>CotsGroup[]</code>	Array of user-defined panel groups
<code>cots.settings</code>	<code>Settings</code>	User preferences
<code>cots.install</code>	<code>{ installedAt, version }</code>	Install metadata

8.2 Persistence Invariants

- The graph is saved *on every panel change* (position, size, creation, deletion).
- `saveGraph` updates `updatedAt` to the current timestamp.
- `loadGraph` returns an empty graph if no stored data exists.
- `clearGraph` removes only the graph key; settings and install metadata are preserved.
- Settings are saved explicitly via the Options page “Save Settings” button.

8.3 Cross-Tab Synchronization

All graph and group mutations are routed through the background service worker, which broadcasts state to every active tab via `chrome.tabs.sendMessage`:

1. User creates/moves/deletes a panel on Tab A.
2. Content script sends `COTS_ACTION` → `PANELS_UPDATED` to background.
3. Background persists to `chrome.storage.local` and broadcasts `GRAPH_UPDATED` to all tabs.
4. Content scripts on Tab B, Tab C, etc. receive the broadcast and merge the incoming graph with their local state (preserving transient state like conversation history and loading spinners).
5. The same cycle applies to groups.

This means:

- Panels created on one tab appear on all other open tabs without reloading.
- Closing and reopening a tab restores the graph from storage.
- Groups and their summaries are always consistent across tabs.

9. Phased Roadmap

The project follows the three-phase architecture described in the original COTs vision:

9.1 Phase 1 — Browser Extension MVP (*Current*)

Feature	Status
Text highlight detection	Implemented
Floating AI context panel	Implemented
Drag, resize, minimize, maximize	Implemented
Source-to-panel edge rendering	Implemented
Graph persistence (local)	Implemented
Panel creation from highlight	Implemented
Follow-up questions	Implemented
Multi-provider AI support	Implemented
Options page (settings)	Implemented
Popup with graph summary	Implemented
Node grouping with correlation summaries	Implemented
Cross-tab state synchronization	Implemented
Real-time persistence across tabs	Implemented
Group management in popup	Implemented
Color-coded group edges	Implemented
Image/link selection anchors	Planned
Multi-range context metadata	Planned
Streaming AI responses	Planned
Conversation history persistence	Planned
Branching (child panels)	Planned
Context tree navigation	Planned
Testing suite	Planned

9.2 Phase 2 — Semantic Graph Layer (*Future*)

- Embedding generation for all panel content
- Semantic clustering of related nodes
- Automatic grouping of redundant or complementary panels
- Contextual memory (panels “remember” related panels)
- LLM-powered node summarisation and tagging

9.3 Phase 3 — Multi-Agent Orchestration (*Future*)

- Specialised expert tabs (one LLM instance per panel, each with a distinct role)
- Reasoning delegation (sub-tasks farmed out to child panels)
- Synthesis nodes that merge findings from multiple panels
- Contradiction detection across sources
- Automated research report generation from the graph

10. Research Applications

10.1 Use Cases

Use Case	How COTs Helps
Literature review	Highlight key claims → panels analyse each → group related claims → AI correlation summary → graph reveals connections
Competitive analysis	Open competitor pages → highlight differentiators → group by competitor → compare panels side-by-side
Exploratory learning	Branch from a core concept into sub-questions → group sub-topics → see how answers thematically connect
Fact-checking	Highlight claims → ask same question of multiple sources → group by source → AI contradiction detection
Systematic review	Highlight findings across papers → group by theme → generate synthesis summary
Code documentation	Highlight API docs → ask usage questions → panels persist as reference

10.2 Research Questions the Project Addresses

1. Does spatial visualisation (graph edges + persistent panels) improve comprehension in multi-source research?
2. Can LLM-powered context tabs reduce context-switching overhead in browser-based research?
3. What graph structures emerge from real research workflows? (tree, lattice, star?)
4. How does the branching depth correlate with research topic complexity?
5. Do AI-generated group correlation summaries accurately capture thematic connections between highlighted content?
6. Does cross-tab panel persistence reduce context fragmentation in multi-source research?
7. Does multi-agent orchestration produce higher quality syntheses than single-agent flat responses?

11. Development

11.1 Commands

```
npm install          # install dependencies (Node 20-24)
npm run dev          # Chrome MV3 dev server with hot reload
npm run dev:firefox  # Firefox MV2 dev server
npm run build        # Chrome MV3 production build
npm run build:firefox # Firefox MV2 production build
npm run package      # Chrome MV3 zip for store
npm run package:firefox # Firefox MV2 zip for store
npm run typecheck    # TypeScript type checking
```

11.2 Environment

OPENAI_API_KEY=sk-... # optional, for local AI testing
FIREFOX_EXT_ID=my-addon@id.org # required for Firefox builds

FIREFOX_EXT_ID is injected into manifest.json as browser_specific_settings.gecko.id during Firefox builds only (Plasmo strips it for Chrome targets).

11.3 Project Structure

```
src/
├─ background.ts          # Service worker
├─ contents/
│   └─ cots-overlay.tsx   # Content script (main overlay)
├─ lib/
│   ├── ai.ts             # AI provider callers
│   ├── browser.ts        # webextension-polyfill re-export
│   ├── id.ts             # UUID generator
│   ├── storage.ts        # Browser storage CRUD for graph, groups, settings
│   └─ types.ts           # All TypeScript types
├─ types/
│   └─ webextension-polyfill.d.ts # Polyfill type declarations
├─ pages/
│   ├── Popup.tsx         # Extension popup
│   └─ Options.tsx        # Settings page
├─ popup.tsx              # Popup entry point
├─ options.tsx            # Options entry point
├─ styles/
│   └─ globals.css        # Tailwind + custom styles
docs/
├─ COTs.pdf               # Original vision document
├─ COTs.typ               # This specification
├─ Plan.md                # Project timeline & weights
├─ architecture.md        # Architecture overview
└─ Languages.md           # Stack summary
```

11.4 Risk Areas

Risk	Level	Mitigation
DOM anchoring on dynamic pages	Very high	MutationObserver + scroll re-sync
Graph synchronisation	High	Debounced updates, viewport versioning
Browser compatibility	Medium	Abstraction layer for storage/API
AI cost scaling	Medium	Token limits, configurable models, local fallback

12. References

Ji, S., Pan, S., Cambria, E., Marttinen, P., & Yu, P. S. (2021). A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE Transactions on Neural Networks and Learning Systems*, 33(2), 494–514.

Li, H., Appleby, G., Alperin, K., Gomez, S. R., & Suh, A. (2025). The Role of Visualization in LLM-Assisted Knowledge Graph Systems: Effects on User Trust, Exploration, and Workflows. *arXiv preprint arXiv:2505.21512*.

Appendix A — TypeScript Type Reference

```
type CotsNodeKind = "selection" | "ai-panel" | "group-summary"
```

```
interface SelectionRect {  
  x, y, width, height: number  
  top, right, bottom, left: number  
}
```

```
interface SelectionAnchor {  
  text: string  
  pageTitle: string  
  pageUrl: string  
  rect: SelectionRect  
  createdAt: string  
}
```

```
interface CotsNodePanel {  
  sourceId: string  
  x, y, width, height: number  
  minimized: boolean  
  maximized: boolean  
  groupId?: string  
}
```

```
interface CotsNode {  
  id: string  
  kind: CotsNodeKind  
  label: string  
  anchor?: SelectionAnchor  
  panel?: CotsNodePanel  
  groupId?: string  
  tabId?: number  
  tabUrl?: string  
}
```

```
interface CotsEdge {  
  id: string  
  sourceId: string  
  targetId: string  
  label: string  
}
```

```
interface CotsGraph {  
  nodes: CotsNode[]  
  edges: CotsEdge[]  
  updatedAt: string  
}
```

```
interface CotsGroup {  
  id: string  
  label: string  
}
```

```

    nodeIds: string[]
    summary?: string
    createdAt: string
    updatedAt: string
}

type CotsActionType =
  | "PANELS_UPDATED" | "PANEL_DELETED"
  | "GROUPS_UPDATED" | "GROUP_DELETED"
  | "SUMMARIZE_GROUP" | "SYNC_REQUEST"
  | "FULL_STATE_REQUEST"

interface CotsAction {
  type: CotsActionType
  payload?: unknown
  sourceTabId?: number
}

type CotsBroadcastType =
  | "FULL_STATE_SYNC" | "GRAPH_UPDATED"
  | "GROUPS_UPDATED" | "GROUP_SUMMARY"

interface CotsBroadcast {
  type: CotsBroadcastType
  payload?: unknown
}

interface FullState {
  graph: CotsGraph
  groups: CotsGroup[]
}

type AiProvider = "openai" | "anthropic" | "google" | "custom"

interface AiRequest {
  selection: SelectionAnchor
  prompt?: string
  conversationHistory?: Array<{ role: "user" | "assistant"; content: string }>
}

interface AiResponse {
  text: string
  model: string
  conversationId?: string
}

interface Settings {
  aiProvider: AiProvider
  openaiApiKey?: string
  openaiModel: string
  anthropicApiKey?: string
  anthropicModel: string
  googleApiKey?: string
  googleModel: string
  customEndpoint?: string
  customModel?: string
}

```

```
systemPrompt: string
maxTokens: number
temperature: number
enableFollowUp: boolean
theme: "dark" | "light" | "auto"
panelWidth: number
panelHeight: number
}
```